

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 - Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 - Articulation (BL3)	Assessment:	Assessment Tool 3 - Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	K. Sushesika	Reg. No.:	192511070
Section / Batch:	BE CSE	Date:	26/08/2026

Code of Conduct

I K. Sushesika (192511070) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: [Signature]

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

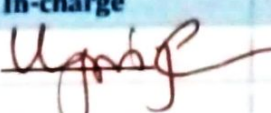
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4 - Assessment

Tool - 3

Name : K-Sushesika

Reg No : 192511070

Code : CSA0305

Course : Data Structures

① Factor	Linear Probing	Separate Chaining
a) Clustering	Causes primary clustering because elements are stored in consecutive slots	No primary clustering. Collided elements are stored in separate chains.
b) Memory Usage	Uses less memory because elements are stored directly in the hash table.	Uses ^{more} memory because extra linked-list nodes and pointers are required.
c) High Load Factor	Performance decreases rapidly as the table becomes full	Performs better because multiple elements can be stored in one bucket.
d) Large-Scale Applications	Suitable when memory efficiency and cache performance are important	More suitable for large-scale applications because it handles collisions and high load factors better.

Linear Probing - Less memory, faster cache access, but suffers from clustering

Separate Chaining - More memory, but better collision handling and high-load performance.

②

Factor	Quick Sort	Merge Sort
Time Complexity	Average : $O(n \log n)$, Worst : $O(n^2)$	Best/Average/Worst : $O(n \log n)$.
Memory-Linked List	Usually $O(\log n)$ recursion stack	$O(1)$ extra if merging is done by changing links.
Cache Behavior	Poor for linked lists because nodes are scattered in memory	Also not ideal, but sequential traversal makes it better suited.
Linked List Performance	less suitable	Highly suitable
Stability	Usually not stable	Stable
Implementation for Linked List	Partitioning is more difficult.	Easy to split and merge using pointers.

For Linked Lists

Merge Sort is preferable.

Why?

- * Takes guaranteed $O(n \log n)$ time.
- * Does not require random access
- * Can merge nodes by changing pointers, requiring very little extra memory.
- * Avoids the difficult partitioning required by Quick Sort.

For Arrays.

Quick Sort is generally preferred for arrays when average-case performance and memory efficiency are important because:

- * It works in-place with $O(\log n)$ average recursion space.
- * Has good cache locality because array elements are stored contiguously.
- * Usually has lower constant overhead than Merge Sort.

Merge Sort is preferred for arrays when:

- * Guaranteed $O(n \log n)$ performance is required.
- * Stability is important.
- * Extra $O(n)$ memory is acceptable.

3)

Factor	Quick Sort	Merge Sort
Time Complexity	Average: $O(n \log n)$, Worst: $O(n^2)$	Best/Average/Worst: $O(n \log n)$.
Memory	In-place; $O(\log n)$ recursion space	For linked lists, can use $O(1)$ extra space apart from recursion.
Cache Behavior	Poorer for linked lists due to pointer access	Better because merging uses sequential traversal.
Linked list	Less suitable	More suitable.

Arrays	Generally preferred due to good cache locality	Requires $O(n)$ space.
--------	--	------------------------

For 10 million linked-list nodes, Merge Sort is preferable because it guarantees $O(n \log n)$, works efficiently with pointers, and needs little extra memory.

For arrays, Quick sort is generally preferred because it is in-place and has excellent cache performance. Merge Sort is chosen when stability and guaranteed $O(n \log n)$ are required.

3)

Factor	Quadratic Probing	Double Hashing.
Clustering Issues	Avoids primary clustering, but many suffer from secondary clustering	Avoids both primary & secondary clustering effectively.
Collision Resolution Efficiency	Better than linear probing, but probe sequence may repeat for keys with the same initial hash value.	More efficient because it uses a second hash function to generate a different probe sequence.
Dense Hash Tables	Performance decreases as the table becomes very full.	Better suited for dense tables because it distributes collisions more uniformly.

Double Hashing is generally better than Quadratic Probing, especially for dense hash tables, because it reduces

ustering and provides more efficient collision resolution.
 Double Hashing > Quadratic Probing > Linear Probing for
 handling collisions in dense tables.

4)

Factor	Best-Case Complexity	Worst-Case Complexity
Meaning	Minimum time an algorithm takes for favorable input	Maximum time an algorithm may take for unfavorable input
Practical Significance	Shows how fast the algorithm can be in ideal situations.	Shows the performance guarantee and possible maximum delay.
Example	Quick Sort: $O(n \log n)$	Quick Sort: $O(n^2)$
Impact on Selection	Useful when inputs are usually favorable	Important when predictable and reliable performance is required.

a) Practical Significance.

* Best case helps understand the algorithm's performance when the input is already favorable.

* ~~Worst case~~ helps identify possible performance problems with difficult inputs.

* For large datasets, worst-case performance can be very important because a poor case may cause a significant increase in execution time.

b) Impact on Algorithm Selection.

* If input conditions are predictable and favorable, an algorithm with a good best/average case may be selected.

* If consistent performance is required, an algorithm with a good worst-case complexity is preferred.

* Example: Merge Sort has $O(n \log n)$ in both best and worst cases, making it more predictable than Quick Sort, whose worst case can be $O(n^2)$.

5)

Factor	Recursive Quick Sort	Iterative Quick Sort
Space Complexity	Uses recursion stack: $O(\log n)$ average, $O(n)$ worst case.	Uses an explicit stack: $O(\log n)$ average, $O(n)$ worst case.
Ease of Implementation	Easier and simpler to write and understand.	More complex because stack management is done manually.
Stack Overflow Risk	Higher, especially with poor pivot selection.	Lower, since recursion is avoided. memory is controlled using an explicit stack.

Space Complexity

* Recursive: Function calls are stored on the call stack. With balanced partitions, space is $O(\log n)$.

* Iterative: Uses a manually created stack to store subarray ranges, avoiding function-call overhead.